The Computer Lifecycle: vmctl, Firecracker, Snapshots, Admission and Updates

A teaching document for Choir's owner, in the Choir learning series. 14 October 2026. Repository: `origin/main` at 980ffaf4e. This document reads code and existing Choir documents only. It had no live access to Node B or staging; every number about the host comes from a dated report or problem document, and it says which one. Mutation class of this document: **green** (docs only). Almost everything it discusses is **red** when changed: vmctl, VM lifecycle, snapshot validity, deployment routing and restart semantics.

Evidence labels used below: **code** (read in the repository at the commit above, with a path), **doc** (an existing Choir document, linked), **upstream** (a Firecracker or Linux fact, as cited by a Choir document that checked it), and **inference** (reasoning that has not been tested; treat it as a hypothesis).

How to read this

Section 1 is a primer: what a Choir computer is as a product, and what a microVM is as a machine. Section 2 is the host control plane, vmctl and vmmanager: who owns which state, how a computer boots, and how the host survives its own restarts. Section 3 draws the state machine of a computer and ties every transition to the restart rule. Section 4 is snapshots and hibernation. Section 5 is admission control and pressure reclaim. Section 6 is the memory and disk model. Section 7 is deploys, refreshes and the LU guest-update design. Section 8 places Firecracker among the other isolation runtimes. Section 9 walks one computer through a day. Then a glossary, five self-check questions, the three files to read, and sources.

Each topic follows the same arc: the general concept first, then how Choir does it (with file paths and function names), then where it is weak (with the incident that showed it), then the options and their trade-offs.

The security side of the same machinery (the VMM running as root, loopback as a credential, side channels) is covered in depth in the computer isolation security review. This document points to it rather than repeating it.

The short answer

1. **A computer is not a VM.** The product object is a persistent computer identified by a stable `ComputerID` and its event chain. A Firecracker VM is one *realization* of it: replaceable machine state. Most lifecycle bugs in Choir are confusions between the two (doc: computer ontology).
2. **vmctl is a reconcile loop over a JSON registry.** Every two minutes it compares what it believes (ownerships) with what is true (Firecracker processes, guest health, host pressure) and acts on the difference. When the registry was blind to processes it had started, the host ran out of memory (code: `internal/vmctl/host_capacity.go`; doc: the 10-10 OOM).
3. **There are two kinds of boot, and the difference is a file.** A boot with a fresh planned-restart marker resumes work; a boot without one is a crash and closes open work. Hibernation today is either a snapshot resume (not a boot at all) or a kill followed by a crash boot (code: `internal/agentcore/planned_restart.go`).
4. **Snapshots already make a wake cost about half a second.** The cost now sits in writing them: each hibernate writes the full 8 GiB guest, which raises host memory pressure enough to trigger the next reclaim (doc: VM density limits).
5. **Admission and reclaim are two halves of one control loop, and today both read a single noisy sample.** Admission reserves the full guest size for every boot; reclaim fires on a 10-second pressure average and can hibernate several computers in one pass without re-reading the signal (code: `internal/vmctl/pressure_reclaim.go`).

6. **A guest holds all the RAM it has ever touched unless it frees it, and Linux does not free page cache.** Free page reporting is on, yet the paper computer still held its whole 8 GiB guest after half an hour (doc: density report, finding 2).
7. **Deploys are push today and should be pull.** A deploy asks each busy computer once and then forgets it. The LU design turns that into a pending state that a single driver retries at quiet moments, with rings, a health gate and forward-only state (doc: LU guest updates).
8. **Firecracker stays; a backend seam goes in.** Every density lever on the list is a Firecracker feature. Cloud Hypervisor earns a place later for copy-on-write restore, live migration and core scheduling (doc: isolation runtimes).

1. Primer: computers and microVMs

1.1 The product object and its realization

The ontology document is short and worth reading twice. Its core sentence: "Choir does not primarily give each user a autoputer. It gives each user a persistent **computer**" (doc: computer ontology). It then writes the computer as a sum:

```
Computer =
  VM/OS/runtime state
  + Dolt/app state
  + source/build state
  + content blobs
  + artifact/provenance graph
  + route identity
```

The engineering consequence is a naming discipline that shows up all over the lifecycle code:

term	what it is	lifetime
computer	`ComputerID` plus its canonical event chain	as long as the user has it
realization	one VM boot: a Firecracker process, a tap, an epoch	minutes to days; replaceable
epoch	the counter that names a realization (`<vm>-epoch-<n>`)	one per boot
machine snapshot	memory plus device state of one realization	one resume, then gone
semantic snapshot	a `ComputerVersion` checkpoint: event head, release, content witness	durable, portable
route	which realization serves the computer's URL	changes on every boot

A realization dying is not the computer dying. The data disk, the event chain, the Dolt workspace and the source tree survive it. The ontology is explicit that "a realization may be replaced without changing `ComputerID`". The restart rule in `CLAUDE.md` follows from this: a crash restart replaces the realization and closes its open obligations, because nothing guarantees those obligations were in a consistent state when the old realization died.

`ComputerVersion` itself is a pair of references, the code and the artifact program (code: `internal/computerversion/types.go`, type `ComputerVersion` with `CodeRef` and `ArtifactProgramRef`). The ontology warns that this tuple "is not by itself a complete restore address": a full restore also needs the event head and the VM-local projection. Keep that in mind when section 7 discusses rollback.

1.2 What a microVM is

A virtual machine is a guest kernel running on real CPU cores in a hardware mode where it cannot touch host memory or devices. Two pieces make it on Linux. **KVM** is the kernel module that runs guest instructions directly on the CPU and traps out ("VM exit") when the guest touches something it may not. The **VMM** (virtual machine monitor) is an ordinary user-space process that opens `/dev/kvm`, allocates guest memory, and emulates the devices the guest sees. The isolation review's section 1.1 explains this in more depth.

A **microVM** is a VM with almost no devices. Firecracker, Choir's VMM, was written by AWS for Lambda and Fargate. It emulates a handful of devices: virtio network, virtio block, a balloon, a serial console and little else. No BIOS, no PCI enumeration of real hardware, no USB, no graphics. That buys two things: a small attack surface, and fast boot (the upstream specification is under 125 ms to start the guest kernel, cited in the isolation-runtimes comparison table).

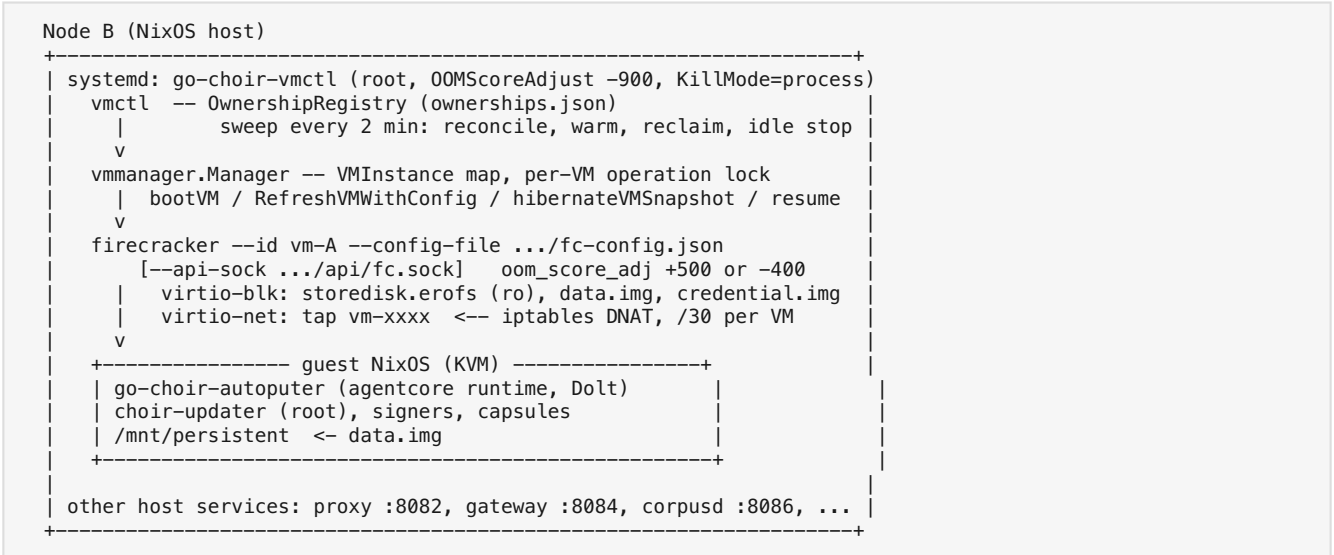
virtio is the device standard: the guest puts requests in shared-memory rings and the VMM processes them. Choir's guest sees these devices (code: `buildFirecrackerConfig`, `internal/vmmanager/manager.go:1563`):

device	host file	guest use
`store` drive, read-only	`/var/lib/go-choir/guest/storedisk.erofs`	the guest's `/nix/store`, shared by every VM
`data` drive, read-write	`<state>/<vm>/data.img`	`/mnt/persistent`: everything the computer owns
`credential` drive	`<state>/<vm>/credential.img`	the one-shot bootstrap envelope and gateway token
`eth0`	a per-VM tap device	the guest's only network
`balloon`	none	free page reporting (section 6)

The kernel and initrd are host files too (`VM_KERNEL_IMAGE`, `VM_INITRD_IMAGE` in `nix/node-b.nix:776-778`). The kernel command line carries the guest's identity and the host service addresses: `vm_id`, `epoch`, `choir.gateway_url=http://<hostIP>:8084`, `choir.vmctl_url`, and so on (code: `manager.go`, the `runtimeArgs` block in `buildFirecrackerConfig`). The guest learns its own IP from the `ip=` parameter, so there is no DHCP.

The guest image is a NixOS system built by microvm.nix with `hypervisor = "firecracker"` (code: `nix/autoputer-vm.nix:247`), but vmmanager launches Firecracker itself rather than using microvm.nix's runner. That matters later: producing a Cloud Hypervisor variant of the same guest is a Nix change, not a new image pipeline.

1.3 The picture



Hold three facts from the picture. vmctl and vmmanager live in one process. Firecracker processes are vmctl's children but survive vmctl's death (`KillMode = "process"`, `nix/node-b.nix`, vmctl unit). And every guest reaches the host services over its own tap, which is why the isolation review calls loopback and the tap addresses a credential.

2. vmctl and vmmanager: who owns what

2.1 Primer: control planes and reconcile loops

A **control plane** decides what should run; a **data plane** runs it. vmctl is Choir's host control plane for computers. The guests are the data plane.

Control planes come in two styles. An **edge-triggered** one reacts to events ("a boot finished", "a stop was requested") and trusts that it saw every event. A **level-triggered** one periodically compares desired state with observed state and acts on the difference, whatever events it missed. Kubernetes controllers are the textbook level-triggered design. The rule of thumb: edge-triggered is fast and fragile; level-triggered is slower and self-healing, provided the observation is complete. When the observation is incomplete, the loop is confidently wrong. Choir learned exactly that on 10 October (section 2.5).

A second idea: **single state authority**. Each fact should have one durable home. If two places record "is this computer running?", they will disagree, and the disagreement becomes a bug class (see standing questions and doc 6 in this series, reasoning about trade-offs).

2.2 How Choir splits the work

vmctl (`cmd/vmctl/main.go`, `internal/vmctl/`) owns the *computer-level* view. Its central type is `OwnershipRegistry`, a map from (user, desktop) to a `VMOwnership` record: the VM id, `ComputerID`, state, epoch, last activity, warmness class, hold flags and the route URL. The registry is persisted to `<state>/ownerships.json` (code: `cmd/vmctl/main.go:39`; `writePersistenceLocked`, `internal/vmctl/ownership.go:657`). The ownership states are `booting`, `active`, `degraded`, `stopping`, `stopped`, `hibernated` and `failed` (code: `ownership.go:50–73`).

vmmanager (`internal/vmmanager/`) owns the *machine-level* view: one `VMInstance` per running realization, holding the process, its PID, its host URL and its boot timeline. vmctl talks to it only through the `VMManager` interface (code: `internal/vmctl/ownership.go:249`), whose verbs are `ReserveBootEpoch`, `BootVM`, `StopVM`, `HibernateVM`, `ResumeVM`, `ReattachVM`, `RecoverVM`, `RefreshVM` and `DestroyVMState`. Optional interfaces add snapshot resume and process control (`snapshotResumer`, `vmProcessController`). That interface is the seam the isolation-runtimes report proposes to keep, with a backend interface added *inside* vmmanager (section 8).

The four ways to start a realization differ in what they keep:

verb	function	kills old process	new epoch	boot kind recorded
boot	`bootVM`, `manager.go:693`	no (refuses a live unmanaged one)	yes	`cold`
recover	`RecoverVMWithConfig`, `manager.go:1399`	yes	yes	caller's
refresh	`RefreshVMWithConfig`, `manager.go:1436`	yes	yes	`refresh`, current deploy's artifacts
snapshot resume	`ResumeVMFromSnapshot`, `snapshot_lifecycle.go:282`	n/a	no: same epoch	`snapshot_resume`

Refresh is recover with one difference: `refreshConfigForCurrentDeploy` (`manager.go:1541`) blanks the stored kernel, initrd, store disk and machine shape, so the boot picks up whatever the current deploy installed. That is how a deploy moves a computer to a new guest image.

2.3 A cold boot, step by step

`bootVM` is the function to read first. Its steps, with the boot-timeline marks it records:

1. Refuse to boot over a live Firecracker process that the manager does not track (`ErrLiveUnmanaged`). That process is a running computer from before a vmctl restart, and booting would kill it mid-work.

2. Allocate a host port; from it derive a /30 subnet in 10.200.0.0 to 10.215.255.255 (`guestAndHostIP`, `allocateHostPortLocked`). Mark `host_port_allocated`.
3. Discard any memory snapshot first (`recordSnapshotFallbackOnBoot`): once a cold boot writes the disk, a snapshot of the old memory would be inconsistent with it.
4. Write the gateway token to `persist/gateway-token` (0600), consume the reserved boot epoch.
5. Create `data.img` if it does not exist (`createDataImage`, `manager.go:2664`), otherwise grow it to the minimum size. Mark `data_image_ready`.
6. Create the credential disk (`createCredentialDisk`). Mark `credential_disk_ready`.
7. Build the Firecracker config, write `fc-config.json`, create the tap and the iptables rules (`prepareVMNetwork`), start the process (`startFirecrackerProcess`, `manager.go:1942`). Mark `firecracker_spawned`.
8. **Release the manager lock**, then wait for the guest's `/health` (`waitForGuestReady`). A guest replaying its event chain answers 503 with an advancing sequence; that extends the wait. Only a stall longer than `ReplayStallTimeout` fails the boot (1,500 s on staging, `nix/node-b.nix:801`).
9. Mark `first_healthy`, fetch the guest's half of the boot timeline, persist the merged record.

Step 8's lock release is a lesson in itself. The comment in the code cites s0-fetch-guest-timeline-deadlock: holding a global lock across a network wait serializes every boot behind the slowest guest. The general rule: **never hold a shared lock across I/O you do not control.** The same rule reappears in the S3 design, where a hibernate's multi-second snapshot write had been made under vmctl's registry lock (S3 failure 19, fixed by calling the manager outside it).

Measured cost (doc: density report §1(d)): a cold boot takes 10.4 to 10.7 s to healthy, of which 2.9 to 3.2 s is creating a new computer's data image (a `git clone` of the platform source plus `mkfs`) and about 7.4 s is the guest booting.

2.4 The sweep: one loop, many duties

`StartIdleSweeper` (`internal/vmctl/ownership.go:1026`) runs every `VMCTL_IDLE_SWEEP_INTERVAL` (2 minutes on staging, `nix/node-b.nix:823`). One pass does, in order:

step	function	what it fixes
1	`refreshTestAccounts`	learns which accounts are test accounts (from the auth DB)
2	`ReconcileVMProcesses` (`host_capacity.go:85`)	every live Firecracker process is accounted for: tracked, reattached, wedged-stopped or reaped
3	`WarmAlwaysOnDesktops`, `WarmUniversalWirePlatformComputer`	boots paid and platform computers that are down
4	`PressureReclaimPlan` / `ReclaimPressureVMs`	hibernates eligible computers under memory or disk pressure
5	`PruneRetention`	deletes state of expired ephemeral and orphaned computers
6	`StopIdleVMs` (`ownership.go:3234`)	hibernates computers idle longer than `VMCTL_IDLE_TIMEOUT` (30 min)

(The second warmer still carries the retired "Universal Wire" name in code; `CLAUDE.md` records the rename to Autopaper as a TODO.)

This is a level-triggered loop: it does not need to have seen a VM start to find it in step 2. The order matters. Reconcile runs first so that every later step acts on a complete registry. Warming runs before reclaim so a protected computer is up before the loop decides what else to evict.

2.5 Surviving vmctl's own restarts

vmctl restarts on every host deploy. If it took its guests with it, every deploy would crash every computer. Three mechanisms prevent that:

- **The guests are not in vmctl's kill set.** `KillMode = "process"` in the vmctl unit means systemd kills only the main process on stop (`nix/node-b.nix`, vmctl unit, with the comment "Let Firecracker child processes survive vmctl process replacement").
- **Loading the registry demotes, then reattaches.** On load, any ownership that was `booting`, `active`, `degraded` or `stopping` becomes `stopped` with `StoppedBy = "vmctl-restart"` (code: `loadLocked`, `ownership.go:624-628`). The startup reattach then adopts each such VM whose guest answers health (`ownership.go:712-745`). A reattach that fails leaves the process running outside the manager, and `ReconcileVMProcesses` retries it every pass.
- **A wedge watchdog bounds "alive but useless".** A guest whose process is alive but which refuses TCP connections to its service port for 5 minutes (`wedgedAfterDefault`, `internal/vmctl/wedge.go:24`) is stopped with a destruction receipt. A busy guest still accepts the TCP connection even when `/health` is slow, so busy is never mistaken for wedged.

Two incidents from 10 October explain why each exists.

The console pipe. Firecracker writes the guest's serial console to its stdout. vmctl used to give it a pipe whose reader was a goroutine *inside the vmctl that launched it*. When that vmctl was replaced on a deploy, the pipe lost its reader. Hours later the guest wrote one line to its console, Firecracker got `SIGPIPE`, and its signal handler deadlocked on its own logger lock while holding the serial device lock. vCPU 0 froze; the guest accepted TCP but answered nothing (doc: firecracker-sigpipe-deadlock). The fix is in `internal/vmmanager/console_file.go`: the console is now a file Firecracker writes directly. "A file needs no reader and outlives every vmctl." The lesson: **a child that must outlive its parent must not depend on anything the parent holds**, including the read end of a pipe.

The blind registry. About five minutes after a deploy, Node B ran out of memory, the kernel's OOM killer fired, and vmctl itself died with SIGBUS. Pressure reclaim had seen the pressure 35 times in 90 minutes but logged `eligible=0 protected=2`: its registry held two ownerships while twelve Firecracker processes ran. The other ten had dropped out of the registry when reattach failed after earlier vmctl restarts, and every reclaim, idle and stop path acts only on the registry (doc: node-b-oom-from-retained-qa-computers). The repair is `ReconcileVMProcesses`: it lists live Firecracker processes from `/proc` (`LiveFirecrackerVMs`, `internal/vmmanager/process_control.go:25`) and makes the registry account for each one. The lesson is the level-triggered one: **a reconcile loop is only as good as its observation.** Observing the registry instead of the world is observing your own beliefs.

The same incident added OOM ordering. vmctl runs at `OOMScoreAdjust = -900`; a new Firecracker process starts at `+500` (`launchOOMScoreAdj`, `process_control.go:22`), and reconcile lowers protected computers to `-400` (`oomScoreAdjProtected`, `host_capacity.go:32`). The kernel kills the highest score first, so under global OOM a test computer dies before the owner's, and vmctl dies last.

2.6 Where it is weak

- **The registry file is written without fsync** (code: `writePersistenceLocked` writes `ownerships.json.tmp` then renames it, with no `Sync` on the file or the directory). On btrfs and ext4 a rename-over usually orders the data first, but a power loss can still lose the newest state or, on some configurations, leave an empty file (*inference*; not observed). The LU design names the same gap for its interim fields (residual R31).
- **Process discovery is by command-line matching** (`firecrackerPIDsForVM`, `firecrackerCmdlineBytesMatchVM`, `manager.go:2175-2249`). It works, but a PID is a weak handle: it can be reused. A `pidfd` or a per-VM cgroup would be exact (*inference*; the isolation review proposes per-VM cgroups).
- **One registry mutex and one sweep.** The density report lists vmctl's single registry lock and its health-probe fan-out among the costs that bind above about 500 VMs (lever L21).

- **The owner's computer was stranded** on 10-10: alive, refusing ARP, protected from reclaim because it was `premium_always_on`, and impossible to restart because the resolve guard kept answering `guest_reattach_pending` (doc: owner-computer-stranded). The wedge watchdog is the repair: protection must be bounded by evidence that the guest can still do work.
- **Every VMM runs as root in vmctl's namespaces and cgroup.** That is the isolation review's central finding (§3.4). For the lifecycle it also means no per-VM memory or I/O accounting: all guests share one cgroup (`MemoryLow = "14G"` on the vmctl unit).

2.7 Options

option	what it buys	cost / risk
fsync the registry file and its directory	crash-durable ownership state	trivial; a few ms per save
move ownership into the platform Dolt (Store A)	one durable authority with history	red; vmctl then depends on Dolt being up to boot anything
per-VM cgroup (density L17)	exact process set, per-VM PSI and memory, `io.max` for snapshot writes	medium; the jailer gives it for free
`pidfd` handles instead of PIDs	no PID reuse races	small, Linux-only
split the registry lock; nftables sets for per-VM rules (L21)	vmctl past ~500 VMs	large; not needed on one host today

3. The state machine of a computer

3.1 Primer: state machines and fates

A lifecycle is easiest to reason about as a state machine: a small set of states, and named transitions between them, each with a precondition and a side effect. Two questions to ask of every transition: *what happens to work in flight?* and *what happens if the transition itself is interrupted halfway?* The second question is where most lifecycle bugs live.

3.2 Choir's states and boot kinds



The registry state says where a computer is. The *boot kind* says what happened to its work. The guest decides the boot kind at start, from the marker (code: `ConsumePlannedRestartMarker`, `internal/agentcore/planned_restart.go:109`). A boot is **planned** only if a marker exists, is readable, carries the binary's work-state epoch (now 2, `CurrentWorkStateEpoch`, `:35`), and is younger than 15 minutes (`plannedRestartMaxAge`, `:26`). Otherwise it is a **crash**. The marker is deleted before it is honored; if it cannot be deleted, the boot is treated as a crash, so a stuck marker can never turn a later crash into a resume.

how the realization changed	marker?	boot kind	fate of open work
deploy refresh after the guest wrote a marker	yes	planned	passivated and resumed
deploy refresh of an old guest without the endpoint, idle	no	crash	closed "interrupted by a restart"
operator refresh (`/internal/vmctl/refresh`)	no	crash	closed
snapshot hibernate and resume	n/a	not a boot	untouched: the process never stopped

how the realization changed	marker?	boot kind	fate of open work
kill hibernate and wake	no	crash	closed
wedge stop, OOM kill, host crash	no	crash	closed
vmctl restart with successful reattach	n/a	not a boot	untouched

Why the asymmetry? `CLAUDE.md` gives the owner's rule: a crash restart never resumes work, because "auto-resume after a crash is how crash-restart loops happen." If a piece of work crashes the computer, resuming it at boot crashes it again. The marker is consumed before resuming, so "a resume that crashes the computer leads to a markerless boot that closes the work: one resume per planned restart, so no loop." Standing intents are the exception: an Autopaper source schedule survives a crash, but the poll in flight does not (code: `internal/agentcore/source_schedule.go`; ruling in `CLAUDE.md`). Doc 2 in this series covers the guest side (the desk runtime).

The lifecycle consequence is blunt: **every kill is expensive.** A kill hibernate, a pressure reclaim before snapshots, an operator refresh, a wedge stop: each closes the computer's work. The paper computer's 14 schedules were closed twice in two days this way (sections 5 and 7).

4. Snapshots and hibernation

4.1 Primer: what a VM snapshot is

A running VM is two things: the contents of guest RAM, and the state of the virtual CPUs and devices (registers, virtqueue positions, timers). A **snapshot** writes both to files. Firecracker writes a small `vmstate` file and a `mem` file the size of guest RAM. Loading it into a fresh Firecracker process and resuming the vCPUs continues the guest exactly where it stopped: same processes, same open sockets (which are now dead on the far side), same page cache.

Important properties, all upstream facts that the S3 design checked (doc: S3 snapshot hibernate design, "Compatibility finding"):

- **Full vs diff.** A full snapshot writes every guest page. A diff snapshot writes only pages dirtied since the last snapshot (with dirty page tracking), or only pages resident in the VMM's memory (with `mincore`). Diff is a developer preview in Firecracker 1.15.1.
- **Memory backends.** `File` maps the `mem` file `MAP_PRIVATE` for the resumed VM's whole life: pages fault in from the file on first touch, so the file must stay until that Firecracker exits. `Uffd` hands page faults to a user-space page server through `userfaultfd`, which can fetch pages from anywhere (compressed, deduplicated, remote).
- **Resume exactly once.** Two resumes of one snapshot share RNG state, identifiers and tokens. Firecracker's documented secure use is one resume. VMGenID, if the guest module is loaded, tells the guest kernel to reseed its random number generator.
- **Same software only.** A snapshot must be loaded on the same Firecracker version and host kernel, with the same tap name and drive paths.
- **The clock lies.** The guest's wall clock continues from the snapshot moment and must be stepped by the guest.

A snapshot is therefore a **machine snapshot** in the ontology's terms: valid only for the same realization, image and hypervisor. It is not a backup and not portable.

4.2 How Choir hibernates

The switch is `VM_HIBERNATE_MODE` (code default `kill`; staging sets `snapshot`, `nix/node-b.nix:813`). In snapshot mode Firecracker is launched with `--api-sock <state>/<vm>/api/fc.sock` instead of `--no-api` (`firecrackerLaunchArgs`, `snapshot.go:290`). `HibernateVM` (`manager.go:989`) then calls `hibernateVMSnapshot` (`snapshot_lifecycle.go:37`), which runs `captureSnapshot`:

```
host (vmmanager)                          guest (agentcore)
----------------                          -----------------
ping API socket; budget check
POST /internal/runtime/hibernate/prepare -->  close admission fence
                                              (only if nothing inside)
                                              busy? -> 409, reopen
                                              renew corpusd capability
                                              sync filesystems
                                          <-- 200 prepared
PATCH /vm {"state":"Paused"}
PUT /snapshot/create  (Full) -> snapshot.partial/{vmstate,mem}
kill Firecracker
commitSnapshot: chmod 0600, fsync, record disk identities,
                write metadata, rename partial -> snapshot/, fsync dir
```

Any failure before the commit falls back to the **kill path**: discard the partial files, kill Firecracker, record `hibernate_kill` with a reason (`guest_busy`, `no_api_socket`, `disk_budget`, ...). The guest-side fence is `prepareHibernate` (`internal/agentcore/hibernate.go:281`); its TTL is 2 minutes (`hibernateFenceTTL`, `:39`), measured on a monotonic clock that a paused guest does not advance, so a host that dies after `prepare` cannot leave the guest closed forever.

Two design moves are worth stealing:

1. **Write to a temporary name, fsync, rename, fsync the parent.** A reader either sees no `snapshot/` directory or a complete one. An interrupted capture leaves only `snapshot.partial/`, which is never loaded and is deleted by the next boot. This is the standard recipe for atomic file replacement on POSIX.
2. **Consume before use.** A wake marks the metadata `consumed=true` and fsyncs it *before* starting Firecracker (`consumeSnapshot`, called in `ResumeVMFromSnapshot`). A crash during the resume then leads to a cold boot, never a second load. It is the same shape as the planned-restart marker: delete the token before acting on it, so the action happens at most once.

The wake path is `startExistingVM` (`internal/vmctl/ownership.go:1329`), which offers a hibernated, unheld computer to `resumeFromSnapshot` before any cold boot. `ResumeVMFromSnapshot` validates the snapshot against a long list of reasons it could be stale (`validateSnapshot`, `snapshot.go:453`): consumed, wrong metadata version, wrong VM or computer or epoch, older than the maximum age, past its credential window, different Firecracker version, different host kernel, a launch-config hash that differs from what a cold boot would use now, missing files, or any disk whose identity (path, size, mtime, inode, device) changed. Any mismatch discards the snapshot and cold boots. This is **fail-closed** validation: when in doubt, take the slower path that is always correct.

Two kinds of epoch appear here, and the S3 design flags the confusion. The **realization epoch** names the boot; a snapshot resume keeps it. The **credential revocation epoch** lives on the event head; every envelope consumption (a boot or a resume) advances it by one and revokes older capabilities. The re-credential slice lets a resumed guest receive a fresh envelope from vmctl in the `resumed` call, so a snapshot is no longer limited by the 30-minute capability life (doc: S3 design, "Re-credential on resume").

Ages and budgets: the code default maximum age is 12 hours (`DefaultSnapshotMaxAge`, `snapshot.go:65`), and staging sets 36 hours (`VM_SNAPSHOT_MAX_AGE=36h`, `nix/node-b.nix:819`, an owner ruling for overnight hibernation). The budget is 48 GiB (`VM_SNAPSHOT_BUDGET_GIB`); `ensureSnapshotRoom` (`snapshot.go:659`) evicts the oldest snapshots first and also requires 16 GiB of free disk after the write.

Measured result (doc: density report §1(d), n=2 wakes): a snapshot wake is 338 to 515 ms to healthy, against 10.4 to 10.7 s for a cold boot. Right after a wake the Firecracker process holds 0.19 GiB, growing to 0.51 GiB at 5 minutes, of which only 0.12 GiB is anonymous: most of it is clean pages mapped from the `mem` file, which the host can drop and read back.

4.3 Where it is weak

- **Every hibernate writes the full guest.** An 8 GiB guest writes an 8 GiB `mem` file: 7.6 to 8.1 s of I/O that pushed host memory PSI to 2.5 and triggered the next pressure reclaim on 14 October (doc: pressure-reclaim-wedged-a-user-computer). btrfs zstd compresses it to about 560 MB on disk, but the write and the page-cache churn happen first.
- **The budget counts the wrong bytes.** It should count real disk bytes, measured with a btrfs extent search. That search needs `CAP_SYS_ADMIN`, which the vmctl unit does not grant (`CapabilityBoundingSet` is `CAP_NET_ADMIN CAP_SYS_PTRACE`, `nix/node-b.nix:724`), so the measure falls back to `st_blocks`, which on btrfs reports the uncompressed size. 48 GiB therefore admits about 5 warm snapshots instead of about 70 (doc: snapshot budget falls back to logical bytes). Note the fallback direction: it overcounts, which is the safe side.
- **Every deploy that touches guest artifacts invalidates every snapshot.** Validation binds the kernel, initrd and store image by file identity, and a deploy rewrites them at stable paths. On 14 October a user's computer was hibernated by pressure at 11:40, a deploy changed the launch config, its wake at 11:43 failed validation with `launch_config_mismatch`, and the fallback cold boot wedged for about 5 minutes (doc: pressure reclaim problem). The wedge window in `wedge.go` is also 5 minutes, which fits the observed outage (*inference*; the problem doc has not yet confirmed the cause from vmctl's journal). For 36 hours after such a deploy, every wake is a cold boot: the density report calls this the "deploy storm" (lever L20).
- **A snapshot is the guest's RAM in plaintext on an unencrypted disk** for up to 36 hours: the gateway bearer, the privacy data key, open documents (doc: isolation review §3.8, finding F12).
- **The S3 decision D3 rested on a premise the density report later corrected.** D3 rejected diff snapshots because `mincore` misses swapped pages and "Node B has swap". The density report points out that guests never swap: the vmctl unit sets `MemorySwapMax=0` and the owner ruled swapping out as service degradation (lever L3, lever L15). That reopens sparse snapshots.

4.4 Options

lever (density report id)	mechanism	gain	risk
L3b budget measure	give the measure `CAP_SYS_ADMIN` via a helper, or rely on L3	~5 to ~70 warm snapshots at once	red; a capability on a root process
L3 sparse snapshots	`snapshot_type: Diff` with `mincore`: write only resident pages	8 GiB write to ~1.2 GiB; hibernate ~1 s; honest `st_blocks`	red; developer preview; wrong if any guest page could swap; a restored VM needs a rebase
L20 snapshots survive deploys	keep versioned guest artifacts until no snapshot needs them; resume on the old release, update at the next idle moment	no deploy storm	red; disk for old artifacts
L18 UFFD page server	a host page server deduplicates and compresses pages across snapshots and prefetches each computer's working set	stable wake latency under load; cross-VM dedup	extra large; a new component
encrypt at rest	LUKS on the host disks; or an encrypting page server	snapshots and disks no longer plaintext	full-disk encryption is cheap; per-snapshot keys are not

The order the density report recommends: L3b first (hours of work, an immediate 14 times more warm snapshots), then the reclaim fix (section 5), then L3, then a shorter idle timeout (L4). That order is a good habit to copy: fix the measurement before you tune against it.

5. Admission control and pressure reclaim

5.1 Primer: admission, reclaim and overcommit

A host has finite memory. Two mechanisms keep it from running out:

- **Admission control** decides whether a new workload may start. It is a gate at the door. Its classic failure is the **check-then-act race**: ten requests read "8 GiB free" at once, each decides it fits, and all ten start. The cure is a **reservation**: admission subtracts what it has promised but not yet seen used.
- **Reclaim** removes running workloads when the host is short. It is a bouncer inside the room. Its classic failure is **oscillation**: it acts on a noisy signal, its own action produces more of the signal, and it acts again. The cure is **hysteresis** (act when the signal crosses a high mark, stop only when it falls below a lower one), a longer averaging window, and rate limits.

Overcommit is admitting more than the sum of maximums because workloads rarely peak together. Every cloud does it. It is safe only if there is a backstop when they do peak together: reclaim, a balloon, or the OOM killer.

The Linux signals involved:

signal	meaning	trap
`MemAvailable` (`/proc/meminfo`)	the kernel's estimate of memory available without swapping, including reclaimable page cache	a host can show 12 GiB available and still stall
PSI `some avg10` (`/proc/pressure/memory`)	percent of the last 10 s in which at least one task stalled waiting for memory	short bursts (a `du`, a `nix build`) spike it
PSI `avg60`, `avg300`	the same over 60 s and 300 s	slower, but much less noisy
`oom_score_adj`	per-process bias for the OOM killer, -1000 to 1000	only orders deaths; prevents nothing
cgroup `memory.low`, `memory.high`, `memory.max`	protection, throttling and hard limits per group	Choir's guests share one cgroup today

5.2 How Choir does it

Tiers. Who matters more is decided by a reviewed file, not by an environment variable on the host. `internal/computertiers/computer-tiers.json` is embedded in vmctl and lists paid accounts (today one: the owner's, "an ordinary paid account") and platform computers (today one: the Autopaper paper computer). `warmnessClassForOwnership` (`internal/vmctl/warmness_policy.go:109`) maps them to warmness classes, with a numeric priority (`warmnessPriority`, `:139`):

class	who	priority	reclaimed?
`test`	test accounts (from the auth DB)	10	first; recent activity does not protect it
`primary`	everyone else	20	yes, after 30 min idle
`public_platform`	legacy platform owner	30	protected
`premium_always_on`	paid accounts	90	never; warmed if down
`critical_protected`	platform computers (the paper)	100	never; warmed if down

Admission. `admitHostMemoryLocked` (`internal/vmctl/host_capacity.go:256`) runs only when a start would *add* a VM. It admits protected classes unconditionally. For the rest it takes one pressure sample and requires `MemAvailable >= floor + guest size`, where the floor is the larger of 4096 MiB and 15% of RAM (`nix/node-b.nix:830-831`) and the guest size is the interactive shape, 8 GiB on staging (`VM_INTERACTIVE_MEM_MIB=8192`; the code default is 2 GiB, `interactiveVMMemSizeMib`, `ownership.go:1656`). A refusal is a `RecoveryRefusal` of kind `host_memory_pressure` with `Retry-After: 60`. Admission is enabled only when pressure reclaim is in `active` mode; otherwise it admits everything.

Reclaim. `annotatePressure` (`pressure_reclaim.go:245`) sets `ReclaimPressure` when memory is short (available below the floor, or memory PSI `some avg10` above 1.0) or the state directory is short of disk. CPU and I/O pressure are reported but never reclaim. When reclaim pressure is on, `pressureReclaimActionCandidates` ranks active computers by protection, then priority, then longest idle, and selects up to `MaxCandidates` (5) unprotected ones. `protectedReclaimReasons` (`:648`) protects a computer that is held, in a protected class, recently active (30 minutes, except test computers), or busy by its own report. Then `ReclaimPressureVMs` (`ownership.go:2730`) hibernates every selected candidate in that pass.

Idle stop. `idleOwnershipCandidates` (`warmness_policy.go:196`) skips held, busy and protected computers. Under `VMCTL_PRIMARY_KEEPALIVE_MODE=under-capacity` (staging, `nix/node-b.nix:824`) it also skips every `primary` computer unless the host is under reclaim pressure. In words: ordinary computers stay running as long as there is room.

The kernel backstop. OOM scores (section 2.5), and `MemoryLow = "14G"` on the vmctl unit, which protects the guests' cgroup from reclaim by other host services.

5.3 Where it is weak: four incidents in five days

1. **10 October, the blind registry.** Covered in 2.5. The control loop saw the pressure but not the processes.
2. **13 October, the paper computer was reclaimed.** At 21:00:10 pressure reclaim hibernated the paper computer. It was idle between its 10-minute polls, so its only protection, "busy", lapsed. Hibernation was still a kill then, so its next wake was a crash boot that closed its 14 schedules, and since API keys never wake a computer, nothing woke it for hours (doc: live-paper-computer-reclaimed-no-computer-tiers). The root cause was that "protection is per moment, not per computer". The repair is the tiers file. Notice what the repair is: a class that belongs to the computer, held in one reviewed place, instead of a transient property sampled every two minutes.
3. **13 October, a new owner waited 20 minutes.** Finished test computers held memory; admission refused the new owner's first boot again and again with `host_memory_pressure`, while nothing reclaimed the idle test computers fast enough to make room (doc: new-owner-waits-while-qa-computers-hold-node-b-memory). Admission and reclaim were not coupled: a refused start did not ask reclaim to make room.
4. **14 October, reclaim wedged a user.** PSI `some avg10` crossed 1.0 because of two `du` scans and two `nix build`s, while 12+ GiB was still available. Reclaim hibernated a real user's computer that no guest needed the memory from. Then the 8 GiB snapshot write of one hibernate pushed PSI to 2.5 and reclaimed a second computer: one hibernate caused the next (doc: pressure reclaim problem).

Read from the code, the cascade has three ingredients:

- the trigger is a 10-second average, the noisiest signal available;
- `ReclaimPressureVMs` acts on one sample and may hibernate up to five computers in a pass without re-reading pressure between them (code: the loop at `ownership.go:2733–2741` calls no sampler);
- the action (an 8 GiB write) produces the signal that triggered it.

That is the oscillation pattern from 5.1, exactly.

And admission has the check-then-act race. It compares one sample with floor plus 8 GiB and records nothing about boots already in flight (doc: density report, finding 6). Today the 8 GiB over-reservation hides the race: each admitted boot is charged far more than it uses. The day admission charges measured use, concurrent wakes would all pass the same check.

A fifth, quieter weakness: **under-capacity keepalive means idle computers never update.** The idle sweeper never hibernates an ordinary computer while the host has room, so an idle test or user computer keeps its old build until something kills it. Pressure reclaim had been hiding this by restarting computers most days (doc: idle-test-computers-keep-old-builds-root-cause). Two policies, each reasonable, combined into a third that nobody chose.

5.4 Options

lever	change	gain	risk
L2 sustained signal	PSI `avg60` plus `MemAvailable` below the floor for 2 to 3 samples; one hibernate in flight; ignore pressure from our own snapshot write; never reclaim a computer active in the last N minutes	ends false reclaims and the cascade	red; slower response to a real spike, which L1 and the OOM order must cover
L1 admission by measured use, with reservations	charge expected working set (wake p95 ~0.6 GiB, cold ~1.3 GiB) plus a shared burst pool; subtract reservations in flight	more computers per host; no race	red; true overcommit, so L2 and memory leases become the backstop
couple admission to reclaim	a refused start asks reclaim to evict a lower class	a new owner never waits on idle test computers	red; a start can evict someone
L17 per-VM cgroups	per-VM `memory.high`, per-VM PSI, `io.max` on snapshot writes	exact signals; one guest's I/O cannot stall the rest	red; touches launch
L11 virtio-mem leases	boot small, hot-plug memory on request, unplug when idle	guest growth becomes a host grant, so admission is exact	red, large; a guest can refuse to unplug

The tension to learn from is protection versus capacity. Every protected class is a computer the loop may never evict, and admission admits protected classes unconditionally. As the paid tier grows, the only thing that bounds it is the OOM killer. The density report's answer is to make each computer cheaper (sections 4 and 6), so that protection costs less, rather than to protect fewer computers.

6. The memory and disk model

6.1 Primer: guest RAM is not host RAM

Three numbers describe a guest's memory, and they differ:

- **Guest RAM size** (`mem_size_mib`): what the guest believes it has. 8 GiB on staging.
- **Host RSS of the VMM**: guest pages the host has actually backed. A page is backed the first time the guest touches it and, by default, stays backed until the VMM exits. The host has no idea which guest pages are "free" from the guest's point of view.
- **Guest use**: what the guest kernel counts as used. Linux fills spare memory with **page cache** (copies of file data), because unused memory is wasted memory. Page cache is not free memory; the kernel drops it only under pressure inside the guest.

So a long-running guest drifts toward host RSS equal to its full RAM size, even if its processes are small. Three tools fight that drift:

- A **balloon** is a guest driver that "inflates" by allocating guest memory and telling the host it may drop those pages. A host-driven squeeze.
- **Free page reporting** is the balloon's passive mode: the guest kernel reports blocks it has freed (2 MiB by default) and the VMM drops them with `madvise(MADV_DONTNEED)`. It returns what the guest frees, never what the guest caches.
- **virtio-mem** hot-plugs and unplugs memory in blocks. The guest starts small and grows on grant.

Disks have their own version of the same gap. A **sparse file** has a logical size (`st_size`) larger than its allocated blocks (`st_blocks`): truncating a file to 32 GiB allocates nothing. Blocks are allocated on first write and, without **discard** (TRIM) support in the virtual disk, are never returned when the guest frees them. On btrfs, `st_blocks` counts uncompressed bytes, so even the "allocated" number overstates real disk use. A **reflink** (`cp --reflink`, `FICLONE`) shares extents between files until one is written: a cheap copy.

6.2 How Choir does it

Shape. Interactive computers are 4 vCPUs and 8 GiB on staging (`VM_INTERACTIVE_CPU_COUNT=4`, `VM_INTERACTIVE_MEM_MIB=8192`, `nix/node-b.nix:791-792`). The comment there records why: a 4 GiB trial on the compacted owner store thrashed and was rolled back (doc: guest store history bloat and memory shape).

Balloon. `firecrackerBalloonConfig` (`internal/vmmanager/balloon.go:17`) adds `{"amount_mib": 0, "deflate_on_oom": true, "free_page_reporting": true}` when `VM_BALLOON_FREE_PAGE_REPORTING=1` (on in staging, `nix/node-b.nix:806`). Never inflated: reporting only. Free page hinting (a developer preview with a documented corruption race) is never sent. The guest loads `virtio_balloon` explicitly (`boot.kernelModules`, `nix/autoputer-vm.nix:240`).

Go memory limits. `internal/gomemlimit/gomemlimit.go` sets a soft Go memory limit from guest `MemTotal`: half of it for the daemon (at least 512 MiB) and one sixteenth for each desk-session worker (at least 256 MiB), by `LimitFor` (`:129`). A soft limit makes Go's garbage collector run and its scavenger return memory to the guest kernel before the heap doubles; it never kills. The same package fixes Dolt's cache budget before Dolt reads the limit: Dolt sizes its caches once, at first use, from the Go limit (doc: LR guest memory return). `GOMEMLIMIT` in the environment always wins, which makes `GOMEMLIMIT=off` the rollback.

Data disk. `createSparseDataImageSeeded` (`manager.go:2723`) creates `data.img` by truncating a new file to 32 GiB (`dataImageSizeMB = 32768`, `manager.go:75`) and running `mkfs.ext4 -d <staging>`, where the staging directory holds a fresh `git clone` of the host's platform checkout at the commit vmctl was built from (`sourceSeedStagingRoot`). The engineering desk's capsule preflight needs that clean checkout. An existing image smaller than the minimum is grown with `truncate` and `resize2fs` (`ensureDataImageMinSize`). Note the file mode: 0644 inside a 0750 root directory; the isolation review asks for 0600.

Credential disk. A tiny ext4 image per realization carrying the bootstrap envelope and gateway token, kept out of the kernel command line so guest children cannot read it from `/proc/cmdline` (`createCredentialDisk`; comment in `buildFirecrackerConfig`).

Recovery quarantine. A cold recovery moves a damaged `data.img` aside (`QuarantineDataImage`, `manager.go:2795`) and keeps the last `VMCTL_RECOVERY_QUARANTINE_RETAINED` completed quarantines per VM, default 2 (`recoveryQuarantineRetained`, `internal/vmctl/cold_recover.go:145`).

Host-side resilience. A guest's calls to host services go through `internal/hostlink`: a refused dial never reached the host, so it is resent with backoff for up to 3 minutes (`RestartWait`). Anything that did reach the host is returned as it was. This is what lets a guest ride through a host service restart (doc: host-deploy-restart-fails-in-flight-guest-turns).

6.3 What the measurements say

From the density report (measured on Node B, 14 October, unless noted):

quantity	value
fresh cold boot, 3 min idle, host RSS	1.21 GiB (guest processes 370 MiB, anon 80 to 132 MiB, page cache ~640 MiB)
after a snapshot wake	0.19 GiB, 0.51 GiB at 5 min (0.12 GiB anon)
owner's computer	2.7 to 4.4 GiB
paper computer, 34 min after a cold boot	7.85 GiB: its whole guest, with free page reporting on
paper computer CPU	1.2 cores on average over 2,042 s, boot replay included
new computer's `data.img`	536 to 539 MB allocated (the seeded checkout plus ext4 metadata)
test computers' images	0.6 to 3.7 GB (median ~0.67 GB)
owner's image / paper's image	7.4 GB / 24.5 GB allocated
snapshot `mem` file on disk	~560 MB of 8 GiB (btrfs zstd)

Two readings matter. First, an idle computer is mostly kernel reservation and page cache; its true private floor is about 0.13 GiB plus kernel overhead. Second, a busy computer drifts to the ceiling because nothing bounds its page cache. The CPU follow-up found that 98% of the paper's guest time is the main daemon, and most of that is unindexed store scans (doc: computer CPU efficiency; doc 1 of this series, the ledger and the store, explains why). Those scans are also what fills the page cache (*inference*: the density report marks "the 640 MiB idle page cache is mostly store reads" as a hypothesis that needs a guest `fincore` read).

6.4 Where it is weak

- **Page cache grows to the ceiling.** Free page reporting cannot return it (density report, finding 2).
- **The data disk only grows.** Firecracker 1.15.1's virtio-blk has no discard, so blocks the guest frees stay allocated. An offline compaction once took the owner's image from 17.7 GB to 4.5 GB (density report, finding 7; store bloat problem).
- **The platform, not computers, is filling the disk.** The artifact store grew 12.6 GB in one day, with garbage collection running in dry-run only (doc: artifact store grows 12 GB a day). The host slab also held 5.9 GiB of `btrfs_inode` entries, likely from the artifact store's file count (density report, finding 4).
- **Comments that no longer hold.** The guest image config and vmmanager both say KSM deduplicates the shared store pages across VMs (`nix/autoputer-vm.nix:271,288`; `manager.go:233`). The density report measured `pages_sharing=0`: Firecracker never marks guest memory mergeable, so KSM is on and does nothing (finding 5). The isolation review and the density report both recommend against turning it on for real, because cross-tenant page sharing is a side channel. Similarly, the `persistentdisk` package still assumes an 8 GiB data image (`DefaultCapBytes`, `internal/persistentdisk/usage.go:10`) and warns at 7 GiB used regardless of the image's real 32 GiB size, so a large but healthy computer like the paper would always read as "warning" (*inference* from the thresholds; not checked against the UI).
- **No per-VM I/O accounting.** Busy computers write Dolt commits through ext4, btrfs zstd and RAID1; per-VM I/O is unmeasured because there is no per-VM cgroup.

6.5 Options, by tier

The density report's ledger has 21 levers. Grouped by what they attack:

attacks	levers	effect (estimates unless marked)
idle RAM	L4 hibernate after 5 to 10 idle minutes; L5 guest page-cache reclaim (DAMON or cgroup `memory.reclaim`)	running set = online users; idle 1.21 to ~0.5 GiB
busy RAM	L5, L6 process budgets (Dolt cache, worker idle budget), L7 store compaction	busy computer 8 to ~3 GiB
shared pages	L10 the Nix store as virtio-pmem with DAX, one host copy	guest store cache to 0; needs a security panel
disk	L8 re-sparsify images offline, or discard (Firecracker 1.17.0); L9 reflinked seed image; L19 artifact GC	~1 GB per dormant user today; ~2,000 users per EPYC host in the medium tier
exactness	L11 virtio-mem, L17 per-VM cgroups	admission becomes accounting, not guessing
rejected	L13 KSM (side channel), L14 huge pages (defeat reporting and balloon), L15 swap (owner ruling: service degradation)	—

The headline the report draws: on the EPYC 7371 with 128 GB, RAM stops being the limit once a busy computer holds about 3 GiB; the 16 cores bind first, at about 20 busy computers (range 15 to 30). Before the cores, model accounts and dollars bind. That is why the CPU work in doc 1 matters to the lifecycle: the cheapest way to fit more computers on a host is to make each busy one use less CPU.

7. Deploys, refreshes and guest updates

7.1 Primer: how fleets get new code

Immutable images. A modern platform ships a whole image (here, a Nix store path) rather than patching machines in place. An update is "boot the new image"; a rollback is "boot the old one". Choir's guest image is one immutable Nix output; `/var/lib/go-choir/guest` is a symlink to it, swapped at host activation (`system.activationScripts.go-choir-guest-image`, `nix/node-b.nix:1093`).

Push vs pull. In a *push* rollout, a central job walks the fleet and restarts each member. In a *pull* rollout, a central record says what the target is, and each member (or a per-host agent) converges toward it when it can. Push is simple and forgets whoever it skipped. Pull is level-triggered (section 2.1): a member that missed one deploy still sees the target on the next pass.

Rings and canaries. Roll a new image to the least valuable members first, watch them for a soak period, and only then widen. A failure halts the rollout before it reaches the members that matter.

Drain. Before restarting a member that is doing work, stop giving it new work, let current work finish within a deadline, then restart.

Forward-only state. If a new binary migrates its data to a new schema, the old binary can no longer read it. Rolling back across a migration corrupts or strands state. The safe rule is forward-only across schema changes: fix a bad migration with a newer build.

The two generals problem. Two parties communicating over a channel that can lose messages cannot both be certain they agree. A host asking a guest "are you ready to be killed?" over HTTP is this problem: if the "yes" is lost, the host does not know whether the guest committed. Designs that need both sides to agree on a moment usually move the decision to one side.

7.2 How Choir deploys today

Host services. A host deploy (CI, "Deploy to Staging") runs `switch-to-configuration switch`, moves each service's binary pointer under `/var/lib/go-choir/services/<name>`, then restarts the services. Until 14 October the switch itself also restarted them, onto the *old* pointer, and the gateway's graceful drain (`TimeoutStopSec = "11min"`, `nix/node-b.nix:975`) held proxy, auth and corpusd down while it finished. The site went down for 3 min 9 s, 4 min 0 s and 11 min 13 s on three deploys that night (doc: deploy-takes-staging-down-during-gateway-drain). The repair sets `restartIfChanged = false` on the five services, so the only restart is the one after the pointer sync, and restarts the gateway last and alone. The next deploy's probe saw 417 of 417 `/health` samples answer 200 (doc: L1 deploy without site outage). The residual: model calls from computers are refused while the gateway drains (2 min 39 s on that deploy), and a 10-second proxy drain caused five 502s on a later one.

Guests. After the host deploy, CI lists candidates from `GET /internal/vmctl/deploy-refresh` and refreshes them one at a time (`max_parallel_refresh=1`, `.github/workflows/ci.yml:1427`). For each one, `deployRefresh` (`internal/vmctl/deploy_refresh.go:254`):

1. skips held, inactive and divergent computers;
2. asks the guest to write its planned-restart marker (`POST /internal/runtime/planned-restart`, `HandleInternalPlannedRestart`, `planned_restart.go:361`); the guest refuses with 409 while busy;
3. on 409, returns "deferred": **the computer waits for the next deploy**;
4. otherwise calls `RefreshVMForDesktop`, which ends in `RefreshVMWithConfig`: kill Firecracker, boot the current image. If the refresh fails, it clears the marker so the marker cannot outlive a restart that did not happen.

In-guest applies. A separate path, the root-owned `choir-updater` (`cmd/choir-updater/main_linux.go`, `internal/updater/updater.go`), already does most of what a careful update needs, for layered releases:

- `Apply` (`updater.go:192`) checks, before any mutation, that the release was built for the booted base image and that the guest store's schema falls inside the release's window (`checkStateCompatibility`, `:426`): "guest store schema N is newer than release's built schema M; refusing stale binary".
- It stages and verifies the release, then swaps the `current` symlink atomically (`swapCurrentPointer`, `:927`: create a temporary symlink, rename it over `current`, fsync the directory).
- It restarts the runtime, probes health, signs health and materialization receipts, and on a failed probe restores the prior release with a signed recovery receipt (`restorePrior`, `:943`).
- The guest's boot wrapper refuses to run a layer built for another base image, and after 3 failed starts in 60 s serves the base runtime instead (the "bootguard", `nix/autoputer-vm.nix:127-180`).

That is a complete small rollout system inside one computer: compatibility gate, atomic pointer swap, health probe, automatic rollback, boot-loop guard, receipts.

7.3 Where it is weak

- **Busy computers miss every deploy.** The paper is busy most of the time, so it was deferred deploy after deploy; on 14 October it took a fix only through an owner-approved manual refresh, a crash boot (LU design §1.6).
- **Nothing retries a deferred computer** between deploys, and docs-only pushes do not deploy at all.
- **An epoch change turned a planned refresh into a crash.** At 05:48 on 14 October the paper finally took a refresh with a marker, but the new binary read work-state epoch 2 and the marker said 1. By design that is a crash boot, which closed all 14 schedules; the paper was silent about an hour (doc: planned-refresh-closes-paper-schedules). The epoch exists only in the marker; nothing persists it in the store.
- **A deploy with every computer deferred failed.** Under `set -eo pipefail`, `grep -lx 'refreshed' | wc -l` exits 1 when nothing matches, and the script died at an assignment (doc: deploy-refresh-all-deferred-fails-deploy). A shell lesson worth keeping: `set -e` turns "zero results" into "failure" whenever a pipeline's first command signals no match.
- **No health gate and no rollback after an image refresh.** CI fails the deploy after the computer is already on the bad build (LU F7).
- **Every deploy that touches guest artifacts invalidates every snapshot** (section 4.3).
- **Idle computers do not update either**, because of keepalive (section 5.3). `CLAUDE.md`'s sentence "they update idle computers and leave busy ones to their next idle moment" was not true on staging when the root-cause document was written.
- **The owner saw a blank screen** during a deploy, probably an `index.html` whose script chunks were not yet served (*hypothesis* in owner-blank-screen-during-deploy).

7.4 The LU design: pull with a deadline

LU (doc: LU guest updates design, round 5, with its adjudication) turns the push into a pull. Its invariants are a compact course in update design:

invariant	rule	principle it teaches
U1 pull	a deploy changes only the target; vmctl's sweep is the only update driver, one computer at a time per host	single driver, level-triggered
U2 no work cut	the host kills a guest only after the guest's final call succeeded: work held, nothing executing, marker durable, sync done	never force busy
U3 planned stays planned	an epoch change runs a registered migration, not a crash	separate "planned" from "compatible"
U4 bounded	per image: one apply, one automatic rollback, K drain attempts, then a visible "update stuck"	bounded retries
U5 recorded	every attempt and outcome is a record on the computer's own chain	audit where the subject lives

invariant	rule	principle it teaches
U6 one authority	pending is *derived* (booted image differs from target), never stored	single state authority
U7 forward-only across state	no automatic rollback across a work-state epoch or store schema	forward-only migrations

The slices, in order:

- **LU1a, host only**: record the booted image identity at every launch; mark a computer pending when it differs from the target; the sweep retries the existing planned-restart endpoint every pass. This reaches today's guests at their quiet moments without any guest change.
- **LU2, guest**: persist the work-state epoch in the store and run migrations at boot, so an epoch change no longer reads as a crash.
- **LU1b, the drain**: defer new work (return `ErrDeferUnprocessed` so held occurrences use no dispatcher slot), admit continuations of runs already executing by lineage, finish within a deadline, then a final call.
- **LU3a, rings**: test computers, then the paper, then the owner, then everyone; a ring opens only when a member of the previous ring validated and none failed; a failure halts the image fleet-wide.
- **LU3b**: probation and one automatic rollback boot.
- **LU5**: one apply path shared with self-development.

The most instructive part is what did *not* converge. LU1b's final call went through four panel rounds, and all five round-4 panelists sent it back. The adjudication's structural assessment names the shared cause: "two processes agreeing, over a call that can be lost, on when a guest may be killed." That is the two generals problem from 7.1. Each patch closed one lost-message case and opened another (the 2-minute fence racing the 2-minute sweep; a lost 200 making the host abort while the guest had already committed; vmctl restarting mid-call). The proposed structural fix moves the decision to one side: after writing its marker, the guest reboots itself, so there is no moment both sides must agree on. That decision is escalated to the owner, which is the `CLAUDE.md` rule "Escalate on dead ends" working as intended.

8. Runtime choices: Firecracker and its alternatives

The isolation runtimes report compares four runtimes as lifecycle choices:

	Firecracker 1.15.1 (now)	Cloud Hypervisor	gVisor	libkrun
runs the NixOS computer as is	yes	yes (a microvm.nix variant)	no	no snapshots
memory snapshots	full; diff (preview); File and UFFD	yes; copy-on-write restore	sandbox checkpoint	no
live migration	no	yes	cold only	no
memory elasticity	balloon, free page reporting, virtio-mem	same, plus ACPI hotplug	n/a	reporting only
disk discard	from 1.17.0	yes	n/a	undocumented
SMT side-channel control	host-level only	core scheduling per VM	no	no

The recommendation: stay on Firecracker for production computers, because every planned density lever is a Firecracker feature; upgrade to 1.17.0 for discard (pairing it with L20 or an announced window, since it invalidates every snapshot); build a small `Backend` interface inside vmmanager now, with Firecracker as the only backend and zero behavior change; and add Cloud Hypervisor second, for the jobs only it does well. gVisor belongs *inside* the computer (capsule children) and beside it (host tools that parse untrusted content), not instead of it. libkrun is a desktop runtime.

Two lifecycle constraints from that report are easy to forget:

- **Snapshots are not portable** between VMMs, or across most versions of one VMM. A computer can change backend only from stopped. Moving a hibernated computer loses its snapshot, which is a crash boot.
- **Never roll back below Firecracker 1.15.1 while PCI is on.** 1.15.1 is the patched release for CVE-2026-5747, an out-of-bounds write in the virtio-PCI transport, and Choir launches with `--enable-pci`.

The seam is already half there: vmctl only knows the `VMManager` interface. The Firecracker-specific code is in about eight places inside vmmanager (`buildFirecrackerConfig`, `firecrackerBalloonConfig`, the launch and restore argument builders, `startFirecrackerProcess`, `fcapi.go`, the version and hash functions, and the command-line matchers).

9. One computer, one day

A walk-through ties the pieces together. Take the paper computer on a quiet day after all of today's fixes are deployed (*inference*: a composed scenario, not a trace).

1. **06:00, deploy.** CI swaps the guest image symlink and restarts host services (gateway last). The paper's guest rides through the gateway drain with `hostlink` resends. CI asks it for a marker; it is mid-poll, so it answers 409 and is deferred. Today it stays on the old build; with LU1a it becomes "pending" and the sweep tries again in two minutes.
2. **06:04, quiet moment.** (With LU1a.) The marker is written; vmctl refreshes it: kill, boot the new image with a new epoch. The guest finds the marker, epoch matches, age under 15 minutes: a planned boot. Open work is passivated and resumed; schedules re-arm.
3. **12:00, a `nix build` on the host.** PSI `avg10` spikes past 1.0. Reclaim looks at candidates. The paper is `critical_protected`, so it is never a candidate. A test computer idle for an hour is hibernated by snapshot: fence, pause, an 8 GiB write. (With L2, the spike alone would not trigger anything.)
4. **12:30, a stranger signs up.** Admission takes one sample: available memory must exceed the 15% floor plus 8 GiB. It does; the new computer cold-boots in about 10 s, 3 s of which is cloning the source seed (L9 would remove that).
5. **19:00, the test computer's owner returns.** vmctl offers the snapshot to `ResumeVMFromSnapshot`. Firecracker version, host kernel, launch config hash and disk identities all match, because no guest-artifact deploy happened since 12:00. vmctl mints a fresh credential envelope; the snapshot is consumed, loaded, told `resumed`; the guest steps its clock, installs the envelope, appends `key_revoked`, and reopens admission. Healthy in about half a second. Had a guest-artifact deploy happened in between, validation would fail with `launch_config_mismatch` and the wake would be a 10-second cold boot that closes the computer's open work.

Every step in that day is a place where one of the incidents above happened for real.

Glossary

term	meaning
Admission control	The gate that decides whether a new VM may start. Choir's is `admitHostMemoryLocked`.
Balloon	A guest driver through which the host can reclaim guest memory; Choir uses only its free page reporting mode.
Boot kind	How a guest classifies its own start: planned (marker present and valid) or crash. A snapshot resume is not a boot.
Check-then-act race	Several actors read the same state, each decides it may act, and all act; cured by reservations or a lock around check and act.
Cold boot	Starting a realization from disk: kernel, initrd, guest init, event replay. About 10 s.
Computer tier	A computer's protection class from `computer-tiers.json`: paid accounts and platform computers are always on.

term	meaning
Crash boot	A boot without a valid planned-restart marker; it closes open work.
Diff snapshot	A snapshot that writes only dirtied or resident pages, not all of guest RAM.
Discard (TRIM)	A disk command that tells the backing store a block is free. Firecracker 1.15.1's virtio-blk lacks it.
Drain	Hold new work, let current work finish within a deadline, then restart.
Epoch (realization)	The counter naming one boot of a computer; a snapshot resume keeps it.
Epoch (credential revocation)	The counter on the event head; every envelope consumption advances it and revokes older capabilities.
Epoch (work-state)	The schema version of a guest's work state (`CurrentWorkStateEpoch`, now 2).
Fail-closed validation	Refusing the fast path on any doubt and taking the slow path that is always correct.
Free page reporting	The guest reports freed memory blocks so the host can drop them. Does not return page cache.
Hysteresis	Different thresholds for starting and stopping an action, so a noisy signal does not make it flap.
KillMode=process	A systemd setting that kills only a unit's main process on stop, so vmctl's Firecracker children survive vmctl restarts.
Level-triggered	A control loop that compares desired and observed state each pass instead of reacting to events.
Machine snapshot	Memory plus device state of one realization; valid only for the same image and hypervisor, resumable once.
Marker (planned-restart)	A file the guest writes before an intentional restart and consumes once at the next boot.
microVM	A VM with a minimal device model, such as Firecracker's.
OOM score adjustment	A per-process bias that orders the kernel OOM killer's victims; it prevents nothing.
Overcommit	Admitting more than the sum of maximums, betting that peaks do not coincide.
Page cache	File data the kernel keeps in otherwise free memory; not counted as free.
Pending (LU)	A computer whose booted image differs from the target image; derived, never stored.
PSI	Pressure stall information: the share of time tasks stalled on memory, CPU or I/O, averaged over 10, 60 and 300 s.
Realization	One VM boot of a computer: process, tap, epoch. Replaceable.
Reattach	A restarted vmctl adopting a Firecracker process that survived it.
Reconcile	Making the registry account for every live VM process (`ReconcileVMProcesses`).
Reflink	A copy that shares disk extents with its source until either is written.
Refresh	Kill and reboot a realization onto the current deploy's guest image, keeping its data disk.
Ring	A rollout stage; a new image reaches ring k+1 only after ring k validated it.
Sparse file	A file whose logical size exceeds its allocated blocks.
UFFD (userfaultfd)	A kernel mechanism that lets a user-space process serve page faults; used for lazy snapshot restore.
virtio	The paravirtual device standard: request rings in shared memory processed by the VMM.
virtio-mem	Memory hot-plug and unplug in blocks, so a guest can start small and grow on grant.
VMM	The user-space process that emulates a VM's devices (Firecracker, Cloud Hypervisor).
Wedge	A guest whose process is alive but refuses TCP on its service port; stopped after 5 minutes.

Self-check

1. A user's computer was hibernated by pressure reclaim at 11:40 and woke at 11:43 as a cold boot instead of a snapshot resume. Name two different validation failures in `validateSnapshot` that could cause this, and say which one the 14 October incident hit and why a deploy caused it.
2. Why does `ConsumePlannedRestartMarker` delete the marker *before* deciding whether to honor it, and what loop would appear if it deleted it afterwards instead?

3. Pressure reclaim saw pressure 35 times on 10 October and reclaimed nothing, then on 14 October reclaimed two computers that no guest needed memory from. Explain both failures in terms of a control loop's observation and its signal.
4. The paper computer held 7.85 GiB of host RSS with free page reporting on. Why did reporting not help, and which two levers in the density report would bring a busy computer toward 3 GiB?
5. LU1b's final call did not converge after four panel rounds. State the underlying distributed-systems problem in one sentence and describe the structural fix proposed instead of a fifth patch.

Answer sketches

1. Candidates: `launch_config_mismatch` (the config a cold boot would build now hashes differently from the snapshot's), `disk_identity_mismatch` (a bound file's size, mtime, inode or device changed), `firecracker_version_mismatch`, `host_kernel_mismatch`, `max_age_passed`. The incident hit `launch_config_mismatch`: the 11:45 goproxy deploy changed the launch config, and validation binds the guest artifacts and config conservatively, so any such deploy invalidates every snapshot.
2. So that the marker can be honored at most once. If the boot honored it and then crashed before deleting it, the next boot would find it again, resume the same work, crash again, and so on: a crash-restart loop. A marker that cannot be deleted is treated as a crash for the same reason.
3. On 10-10 the observation was incomplete: the registry held 2 of 12 running VMs, so reclaim had no eligible candidates (repaired by reconciling against `/proc`). On 14-10 the signal was wrong: PSI `avg10` spiked from `du` and `nix build` while 12+ GiB was available, and the hibernate's own 8 GiB write raised PSI again, so the action fed its trigger. The fix is a sustained signal (`avg60` plus low `MemAvailable`), one hibernate in flight, and ignoring self-caused pressure (L2).
4. Reporting returns only pages the guest has freed; the guest's page cache (store reads) is not free, so it grows to the 8 GiB ceiling. L5 (guest page-cache reclaim with DAMON or `memory.reclaim`) and L6 (Dolt cache and worker budgets), with L7 compaction behind them.
5. Two parties must agree on a moment (when the guest may be killed) over a channel that can lose messages, which no protocol can guarantee. The proposed fix removes the agreement: after writing its marker the guest reboots itself, so only one side decides.

Read these three files

1. **`internal/vmmanager/manager.go`**, starting at `bootVM` (line 693) and reading through `buildFirecrackerConfig`, `launchFirecracker` and `startFirecrackerProcess`. Look for what each step writes to disk, where the manager lock is released, and which comments still describe old behavior.
2. **`internal/vmctl/ownership.go`**: `StartIdleSweeper` (line 1026) for the order of the control loop, `loadLocked` and the reattach that follows for restart survival, and `startExistingVM` (line 1329) for the wake path: reattach, admission, snapshot resume, recover, boot. Then read `internal/vmctl/host_capacity.go` and `pressure_reclaim.go` beside it.
3. **`internal/vmmanager/snapshot_lifecycle.go`**: `captureSnapshot` and `commitSnapshot` for the atomic write and the kill fallback, `ResumeVMFromSnapshot` for consume-before-load and the credential issuer. Read `validateSnapshot` in `snapshot.go` alongside it and list every reason a snapshot can be refused.

Sources and confidence

- **Built on:** computer ontology; VM density limits; S3 snapshot hibernate design; LU guest updates design and adjudication; L1 deploy without site outage; LR guest memory return; LR provisional scaling cost; isolation runtimes; computer isolation security review §3.3, §3.4, §3.8; computer CPU efficiency; and the problem documents linked in sections 2, 4, 5, 6 and 7.
- **Read in code at 980ffaf4e:** `internal/vmmanager/` (`manager.go`, `snapshot.go`, `snapshot_lifecycle.go`, `fcapi.go`, `balloon.go`, `process_control.go`, `console_file.go`, `receipts.go`), `internal/vmctl/` (`ownership.go`, `host_capacity.go`, `pressure_reclaim.go`, `warmness_policy.go`, `deploy_refresh.go`, `wedge.go`, `cold_recover.go`), `internal/computertiers/`, `internal/agentcore/planned_restart.go`

and `hibernate.go`, `internal/updater/updater.go`, `internal/gomemlimit/`, `internal/hostlink/`, `internal/persistentdisk/`, `internal/computerversion/types.go`, `internal/platformrelease/types.go`, `nix/node-b.nix`, `nix/autoputer-vm.nix`, and the deploy-refresh step of `.github/workflows/ci.yml`.

- **Numbers:** every measured number comes from a dated report or problem document named beside it. None was measured for this document.
- **Inference, marked in the text:** the durability gap of `ownerships.json`; PID reuse as a risk; the 5-minute wedge window explaining the 14 October outage; page cache being mostly store reads; the `persistentdisk` warning firing for large healthy images; the section 9 scenario.
- **Code and comments that disagree:** the KSM comments in `nix/autoputer-vm.nix` and `manager.go` against the measured `pages_sharing=0`; `StateHibernated`'s comment in `manager.go` ("a future capability") against the snapshot path that now exists; the vmctl unit's `MemoryLow` comment ("3 always-on 4 GiB guests") against the 8 GiB interactive shape; the S3 design note's 12-hour maximum age (the code default) against staging's 36 hours.
- **Not covered:** the guest runtime's restart handling in depth (doc 2), the security of the VMM boundary (the isolation review), Store A event heads and credential issuance internals (doc 1 and doc 3), and the macOS desktop runtime.